

PLANEJAMENTO LÓGICO – DEVQUEST (TASKTRACKER GAMIFICADO)

1. APRESENTAÇÃO E ENTENDIMENTO DO PROBLEMA

A eficácia de uma solução de software é diretamente proporcional à robustez de sua fundação lógica. No ciclo de vida de desenvolvimento, o planejamento estruturado transcende a mera documentação: ele atua como um instrumento estratégico de governança para mitigar riscos de execução, evitar paralisia por análise e reduzir o retrabalho técnico. Ao decompor requisitos complexos em fluxos de dados previsíveis antes da codificação, garantimos uma arquitetura escalável e alinhada às expectativas dos *stakeholders*.

1.1 Identificação do Autor

Abaixo, os dados da responsabilidade técnica deste projeto:

Campo	Detalhes
Aluna	Gabrielle Keiko
Curso	Ciência de Dados e Machine Learning
Instituição	UniCEUB
E-mail	gabrielle.keiko@sempreceub.com

1.2 Definição do Problema Central

Estudantes e desenvolvedores enfrentam, rotineiramente, uma sobrecarga mental que compromete a produtividade. A falta de clareza na hierarquização de prioridades e a ausência de recompensas tangíveis geram ansiedade e perda de engajamento. Sem uma organização técnica que converta obrigações em metas visíveis, o processo de gestão de tarefas torna-se

um fardo cognitivo. O projeto visa mitigar esses pontos críticos, substituindo o estresse administrativo por um ecossistema técnico organizado e motivador.

1.3 Proposição da Solução: O Ecossistema DevQuest

O **DevQuest** é um sistema de TaskTracker gamificado projetado para transformar o fluxo de trabalho em uma jornada de progresso. A solução utiliza o conceito de "*Missões*" para categorizar tarefas, atrelando a conclusão de cada item ao ganho de XP (*Experience Points*). Este motor de engajamento é calibrado pela prioridade da missão, incentivando a resolução de desafios de maior impacto. Para garantir a viabilidade técnica desta lógica, é imperativo detalhar os parâmetros de dados que alimentam o sistema.

2. DECOMPOSIÇÃO DO PROBLEMA E DADOS DO SISTEMA

A definição rigorosa de entradas, saídas e regras de negócio é fundamental para assegurar a integridade dos dados e a escalabilidade futura do sistema. Em uma arquitetura orientada a resultados, a governança dos dados impede estados inconsistentes e garante que as métricas de progresso sejam auditáveis e precisas.

2.1 Fluxo de Dados (Entradas e Saídas)

O sistema opera com os seguintes vetores de informação:

Entradas:

- **Título da Missão:** Identificador em formato Texto.
- **Prioridade:** Classificação categórica (Baixa, Média ou Alta).

Saídas:

- **Detalhamento das Missões:** Listagem contendo título, prioridade, status (*'Pendente'* ou *'Concluída'*) e o respectivo valor de XP.
- **Painel Geral de XP Acumulado:** Somatório total da experiência adquirida pelo usuário.
- **Sistema de Mensagens de Feedback:** Notificações de Sucesso (ex: salvamento concluído) ou Erro (ex: campos obrigatórios ausentes).

2.2 Regras de Negócio e Governança do Sistema

As diretrizes abaixo regem a consistência lógica do ecossistema:

Regra	Descrição	Impacto na Consistência
1. Obrigatoriedade	O preenchimento do Título da Missão é mandatório.	Evita registros órfãos ou sem identificação no repositório.
2. Restrição de Prioridade	Entrada restrita aos valores: Baixa, Média ou Alta.	Garante a padronização para cálculos automáticos de recompensas.
3. Atribuição de XP	XP (<i>Experience Points</i>) automático: Baixa=10 XP, Média=25 XP, Alta=50 XP.	Elimina a subjetividade e padroniza o sistema de progressão.
4. Unicidade de Conclusão	Alteração para ' <i>Concluída</i> ' é irreversível e soma o XP ao total.	Assegura que o progresso não sofra manipulações ou duplicidade.

A aplicação estrita destas regras provê a estabilidade necessária para a execução do algoritmo, que servirá como planta baixa para a codificação.

3. RECEITA LÓGICA DO SISTEMA (ALGORITMO EM PORTUGUÊS)

O pseudocódigo atua como a ponte vital entre o requisito de negócio e a implementação técnica, facilitando a validação da lógica por *stakeholders* não técnicos e acelerando o desenvolvimento. Neste projeto, utilizaremos o Google Gemini como ferramenta de apoio para acelerar a tradução desta lógica em código limpo e seguro.

3.1 Algoritmo de Cadastro de Missões

1. Iniciar o formulário de entrada de dados.
2. Solicitar o Título da Missão.
3. Solicitar a Prioridade (Baixa, Média ou Alta).
4. Validar se o Título foi preenchido. Se vazio: exibir mensagem de erro e retornar ao passo 2.

5. Atribuir o valor de XP com base na prioridade selecionada (10, 25 ou 50).
6. Definir o status inicial da missão como "Pendente".
7. Persistir os dados no repositório de missões.
8. Exibir mensagem: *"Missão registrada com sucesso!"*.

3.2 Algoritmo de Visualização e Progressão (Conclusão)

1. Inicializar o sistema carregando o valor total de XP (*Experience Points*) acumulado do repositório.
2. Acessar a funcionalidade de listagem de missões.
3. Verificar se a lista de missões contém registros. Se lista vazia: exibir *"Nenhuma missão disponível no momento"*.
4. Listar todas as missões disponíveis (Título, Prioridade e Status).
5. Permitir que o usuário selecione uma missão com status "Pendente".
6. Validar o comando de conclusão.
7. Alterar o status da missão selecionada para "Concluída".
8. Executar atualização aritmética: somar o XP da missão concluída ao valor carregado no Painel Geral.
9. Persistir o novo valor total de XP no armazenamento de longo prazo.
10. Exibir feedback visual de sucesso e o novo saldo de progresso.

Esta lógica estruturada será suportada por uma arquitetura de versionamento profissional no GitHub.

4. MAPEAMENTO ARQUITETURAL PARA GIT E GITHUB

O versionamento de código desempenha uma função estratégica no gerenciamento de ativos digitais. Uma árvore de diretórios bem planejada e o uso correto do Git facilitam a colaboração em equipes de dados e garantem que o histórico de evolução do projeto seja transparente, seguro e auditável.

4.1 Estrutura do Repositório (Árvore de Diretórios)

A organização dos arquivos seguirá o padrão técnico abaixo:

```
devquest-tasktracker/  
├── README.md           # Apresentação e guia do projeto no GitHub  
├── .gitignore          # Arquivo para ignorar arquivos temporários/locais  
├── docs/  
│   └── planejamento_logico.pdf # Documento oficial de planejamento da Fase 1
```

```
└─ src/  
  └─ main.py          # Código-fonte principal do programa Python
```

4.2 Filosofia de Versionamento e Aprendizado Técnico

A adoção de commits semânticos, *branches* e um histórico de versões organizado é um requisito de excelência para qualquer desenvolvedor moderno. Esta metodologia facilita a integração contínua do projeto ao **PortfolioHUB** (plataforma central de entregas da UniCEUB), transformando um esforço acadêmico em um ativo técnico compartilhável. Durante a implantação, o Google Gemini será utilizado para auditar a segurança e garantir a conformidade com as melhores práticas de versionamento, assegurando que o código final seja não apenas funcional, mas profissional.

5. LINK DA APRESENTAÇÃO EM VÍDEO

A demonstração visual a seguir constitui a prova de conceito do planejamento lógico detalhado, validando a funcionalidade do sistema e a eficácia das regras de negócio implementadas.

5.1 Repositório e Demonstração

Link da apresentação de 5 minutos (YouTube):

https://www.youtube.com/watch?v=COLE_O_SEU_LINK_AQUI